Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #3| Código en Lenguaje C

Ingeniería en Desarrollo de Software



TUTOR: Sandra Lara Devora

ALUMNO: Edgar Enrique Cuamea Ochoa

FECHA: 5 de Agosto del 2024

Indice.

Introducción. 3

Descripción. 4

Justificación. 5

Desarrollo. 6

Código calculadora primos. 7

Calculadora números pares. 9

Calculadora números inversos. 11

Conclusión. 13

Referencias. 14

Introducción.

En esta tercera actividad escribiremos nuestro código en lenguaje C, para ello usaremos como base las actividades anteriores donde escribimos un algoritmo que diferencie los números primos de los no primos, uno donde diferencie los números pare de los impares y un ultimo donde nos devuelva los números inversos en un seudocódigo utilizando lenguaje natural para comprender el algoritmo, después convertimos ese seudocódigo en diagramas de flujo para explicar su funcionamiento para posteriormente escribirlo en lenguaje c explicando así su funcionamiento en un compilador, en este caso usando la página web de programiz ya que además de ser online, es un compilador que no tenemos que descargar y podremos usarla desde nuestro teléfono por lo que nos será sencillo utilizarla si no tenemos computadora, así que utilizando como base los seudocódigos de las actividades anterior desarrollaremos las calculadoras en esta pagina web además de documentar el funcionamiento de las diferentes calculadoras que codificaremos en lenguaje C.

Descripción.

Como vimos anteriormente es las actividades realizadas utilizamos el lenguaje natural para realizar nuestro seudocódigo en la aplicación de PSeint para después pasarlo de manera grafica a diagramas de flujo, esto para las 3 calculadoras que hicimos, así que teniendo como base estas actividades anteriores veremos como codificarla en lenguaje c por lo que tenemos que tener un buen control sobre la sintaxis o escritura que estaremos utilizando ya que un simple error de carácter o un valor que omitamos en nuestro código compilado hará que nuestro código presente errores, en este aso hay que tener en claro como funcionan las compuertas logias y como se representan en lenguaje c en este caso "&&" representa el "y" lógico, "||" representa el "ò" lógico y "!" representa el no, por lo que teniendo esto en cuenta solo tendríamos que traducir el seudocódigo ya escrito a lenguaje c lo que serà mas fácil si sabemos ingles ya que es esta incluido en algunas palabras del lenguaje c y no solo símbolos para ayudarnos a comprender el algoritmo así que si tenemos un buen control de la sintaxis y comprendemos como utilizar las variables y conectores lógicos no debemos tener errores en el código ya que un solo error hará que el código no funcione.

Justificación.

Utilizaremos el compilador online programiz, ya que es una pagina web que nos ayudara a realizar la actividad ya que al ser online y fácil de usar sin pagar subscripciones y sin necesidad de descargar algo será más fácil trabajar en ella desde diferentes dispositivos, ya sea en computadora o desde un teléfono móvil, además de que tiene distintos lenguajes de programación incluidos, desde C, C++, C#, Python, etc. Lo que lo hace una buena opción para utilizar, utilizaremos como base las actividades anteriores utilizando el seudocódigo de las 3 calculadoras que escribimos para traducirlas en lenguaje C en este compilador online, pues utilizando como referencia los seudocódigos ya establecidos nos será más fácil la comprensión y traducción de los algoritmos que previamente realizamos y convertirlos en el compilador a lenguaje C ya que no tendremos que diseñar los algoritmos desde la primera calculadora y solo tendremos que traducirla para una mejor comprensión de los algoritmos y aprender a usar el lenguaje c así como su sintaxis escrita.

Desarrollo.

Dicho anteriormente solo traduciremos el seudocódigo a lenguaje C por lo que necesitamos saber que símbolos utilizaremos, ya que estamos familiarizados con los conectores lógicos, solo debemos traducirlos a símbolos tal que “&&” representa el “y” lógico, “||” representa el “ò” lógico y “!” representa el no, así como el uso de las librerías ya que sin ellas no tendremos algunas de las funciones que necesitamos como el uso de las variables booleanas que solo tienen 2 estados, verdadero o falso, ya sean palabras como “printf” o “scanf” que son funciones para escribir en pantalla y leer alguna entrada de datos y ya que son palabras en ingles podemos asociarlas como escanear función o imprimir función, lo que hará que la comprensión sea mas fácil como las palabras do while, if, else, int, for, etc., así como el uso de las llaves y paréntesis y símbolos que son esenciales para los algoritmos en estas operaciones matemáticas traduciéndolas en lenguaje c.

Código calculadora primos.

```c
#include <stdio.h>
#include <stdbool.h>
int main() {
    bool otro = true;
    while (otro) {
        int n, contador = 0;
        printf("ingresa un numero: ");
        scanf("%d", &n);
        if (n == 0) {
            printf("ingresa un numero que no sea 0: ");
            scanf("%d", &n);
        }
        for (int d = 1; d <= n; d++) {
            if (n % d == 0) {
                contador++;
            }
        }
        if (contador == 2) {
            printf("el numero ingresado es un numero primo\n");
        } else {
            printf("el numero ingresado no es un numero primo\n");
            printf("quiere ingresar otro numero? (s/n): ");
            char respuesta;
            scanf(" %c", &respuesta);
            if (respuesta == 'n') {
                otro = false;
            }
        }
    }
    return 0;
}
```

Este es nuestro código en c de la calculadora de primos donde las primeras 2 líneas de nuestro código incluyen las bibliotecas donde stdbool.h es la bibliotecas o librerías de las funciones

booleanas donde una variable tiene 2 estados, verdadero o falso, en la tercera línea iniciamos nuestro código main donde definimos la variable otro como booleana con un estado verdadero donde ejecuta el código que mientras este sea verdadero defina la variable n y contador igual a 0 e imprime un texto donde nos pide ingresar número y lee un numero entero decimal y lo guarda en la variable n, después ejecuta un bucle donde definimos la variable d igual a 1 y mientras d sea menor o igual a la variable n este sumara 1 a la variable d, si el residuo de n entre la variable d es igual a 0 entonces pasa a sumar 1 al contador, así que si el contador es igual a 2 el numero es un numero primo, si no, el numero no es primo por lo que dependiendo de este contador imprime una de las dos opciones y pide si deseamos ingresar otro número, en caso de que así sea nos devolverá a la línea 0 donde ejecutaremos el código nuevamente ingresando otro numero y finalizaremos al escribir la letra n si no queremos ingresar otro número.

```
ingresa un numero: 5
el numero ingresado es un numero primo
ingresa un numero: 6
el numero ingresado no es un numero primo
quiere ingresar otro numero? (s/n): s
ingresa un numero: 8
el numero ingresado no es un numero primo
quiere ingresar otro numero? (s/n): n


=== Code Execution Successful ===
```

Calculadora números pares.

```c
#include <stdio.h>
#include <stdbool.h>
int main () {
    bool otro = true;
    while (otro) {
    int pares=0,impares=0,contador=1;
    while (contador<=10) {
        int n;
        printf("ingrese un numero\n");
        scanf("%d", &n);
            contador++;
        if (n%2==0){
            pares++;
        printf("%d es un numero par\n",n); }
        else { impares++;
        printf("%d no es un numero par\n",n);
    }
}
printf("%d son numeros par\n", pares);
        printf("%d son numeros impares\n", impares);
        printf("quiere ingresar otro numero? (s/n):");
        char respuesta;
        scanf(" %c", &respuesta);
        if (respuesta == 'n') {
            otro = false;
        }
    }
    return 0;
}
```

En este código se puede ver que mientras la variable booleana (otro) sea verdadera esta inicializara las variables pares e impares en 0 y un contador en 1 donde mientras el contador sea menor o igual a 10 agrega una variable n, se inicia el código conde nos pide ingresar un numero, lo lee y lo guarda en la variable n y el contador pasa a sumar 1, si el residuo de el numero que

ingresamos entre 2 es igual a 0 entonces es un numero par e imprime el mensaje de que es un numero par, si no es así entonces imprime el mensaje de que no es un numero par, después nos dice cuantos de esos 10 números que agregamos son par y cuantos son impar, así que al final nos pide si queremos ingresar otro número, si es así nos regresa a la primera línea, si no, la variable otro se convierte en falsa y finaliza el código.

```
ingrese un numero
8
8 es un numero par
ingrese un numero
10
10 es un numero par
ingrese un numero
3
3 no es un numero par
ingrese un numero
5
5 no es un numero par
ingrese un numero
7
7 no es un numero par
ingrese un numero
9
9 no es un numero par
ingrese un numero
0
0 es un numero par
ingrese un numero
31416
31416 es un numero par
6 son numeros par
4 son numeros impares
quiere ingresar otro numero? (s/n):n

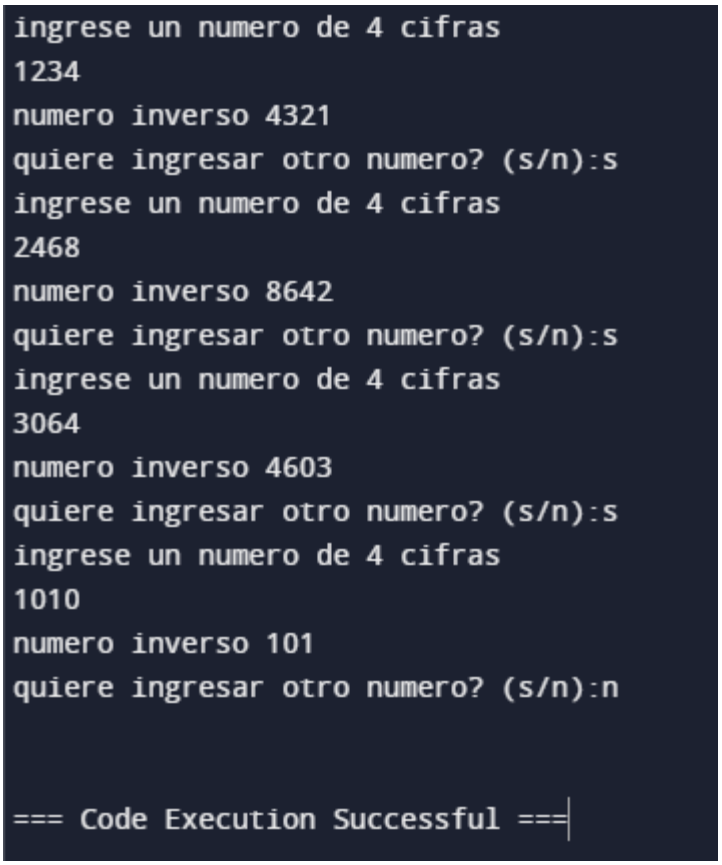

=== Code Execution Successful ===
```

Calculadora números inversos.

```c
#include <stdio.h>
#include <stdbool.h>
int main() {
    bool otro=true;
    while (otro){
        int n, ni=0, d;
    printf("ingrese un numero de 4 cifras\n");
    scanf("%d", &n);
    while(n != 0){
    d=n%10;
    ni=(ni*10)+d;
    n=n/10;
    }
    printf("numero inverso %d\n", ni);
     printf("quiere ingresar otro numero? (s/n):");
        char respuesta;
        scanf(" %c", &respuesta);
        if (respuesta == 'n') {
            otro = false;
        }
    }
    return 0;
}
```

En esta ultima calculadora tenemos que ingresar un numero de 4 cifras y que este lo regrese de manera inversa para eso se inicia una variable booleana (otro) en verdadera, mientras esta sea verdadera agrega otras variables (n, ni, d) donde n es el numero ingresado, ni es el numero invertido y d es una variable temporal, después nos pide ingresar un numero de 4 cifras y lo guarda en la variable n y mientras n no sea 0 la variable d tendrá el valor de el residuo de n entre 10,

después la variable ni tendrá el valor de ni por 10 mas la variable d, por lo que n adquiere el valor de n entre 10, y ejecuta de nuevo el comando while siempre que n sea diferente a 0, nos imprime nuestro numero inverso guardado en la variable ni y nos pide si queremos ingresar otro número, si es así, nos devolverá a la primera línea, si no, cambiara el valor de (otro) a falso finalizando el código.



Conclusión.

Tendremos que tener una buena comprensión de la sintaxis a la hora de escribir códigos ya que al equivocarnos en alguna línea hará que el código no se ejecute bien, el compilador te dice en que línea se encuentra un error y donde se espera que este en ese error, en ciertos casos tuve algunos donde no sabia donde estaba el error hasta que vi que me faltaban símbolos que no tuve en cuenta al realizar el código, así como el uso de las variables que tengamos y donde ponerlas, así como las llaves que usaremos ya que es fácil equivocarse, lo que nos ayuda a tener un poco mas de control el saber donde están esos errores y cómo podemos solucionarlos para que nuestro código se ejecute bien, ya sea en una calculadora o en algún circuito o programa que estemos diseñando, así como aprender a usar los conectores lógicos para los algoritmos.

Referencias.

Hackemi. (s. f.-c). *GitHub - Hackemi/Calculadoras: Actividades entregadas en PDF de la introduccion al desarrollo de software*. GitHub. https://github.com/Hackemi/Calculadoras/tree/main

See my C program in action. (s. f.). https://www.programiz.com/online-compiler/83aiUvEtVJkt9

See my C program in action. (s. f.-b). https://www.programiz.com/online-compiler/1JePZIUuW7Ala

See my C program in action. (s. f.-c). https://www.programiz.com/online-compiler/793bVYSH3ifzQ